Project Proposal

HPC Regression Testing Platform

Submitted to:

William Edsall

Dow HPC

Prepared by:

Team DOW-1-26

Shree Patel	Project Lead
Kayleen Nordyke	QA/Validation Lead
Shawn Schulz	Tech Lead
Brent Thorne	Docs & DevOps Lead

February 2026

DRAFT

Executive Summary

Dow's scientific computing environment supports critical R&D in chemistry, materials science, molecular simulation, and advanced analytics. Evolving IT, industry, and government security standards now mandate more frequent software and system updates, increasing the likelihood of unintentionally degrading the performance of key scientific solvers. Like many organizations operating HPC[1] environments, Dow currently relies on a combination of expert judgment and semi-manual validation to identify regressions. While effective, this approach is time-intensive and difficult to scale, creating an opportunity for a general, execution-agnostic testing framework that amplifies the capabilities of HPC teams.

To address this challenge, our team will develop a lightweight, modular regression-testing platform that automatically executes scientific workloads, captures performance metrics, detects anomalies, and visualizes historical trends. The platform is execution-agnostic, enabling solver scripts to define their own run logic while the system provides standardized configuration, logging, and metric extraction. Results will be stored in a lightweight database and presented through a web interface that highlights recent runs, long-term trends, and detailed performance data. Although this project is sponsored by Dow, the framework is designed to be broadly applicable across heterogeneous HPC environments.

The project follows a structured build–validate–iterate approach, supported by weekly sponsor and faculty check-ins to ensure alignment, rapid feedback, and early assumption testing. Team roles span project leadership, technical architecture, validation strategy, and documentation, enabling coordinated progress across design, implementation, and delivery.

Key deliverables include a functional prototype test harness, a modular configuration system for resources, systems, solvers, and tests, a performance-tracking dashboard, comprehensive documentation, and a public GitHub repository suitable for long-term community maintenance. The resulting platform will reduce manual testing burden, improve confidence during system updates, and help Dow—and other HPC operators—maintain stable, reliable scientific computing performance over time.

[1] High-Performance Computing; large-scale computational systems used for scientific and engineering workloads.

DRAFT

Contents

Executive Summary **1**

1 Objectives **4**
1.1 Project Objectives 4
1.2 Scope and Constraints 4
1.3 Expected Outcomes and Deliverables 5
1.4 Value Proposition of Deliverables to Sponsor 5

2 Technical Approach **5**
2.1 Overall Design Process 5
2.2 Concept Generation 6
2.3 Analysis and Modeling 6
2.4 Experimental Methods 7

3 Project Management **7**
3.1 Work Breakdown and Phases 7
3.2 Schedule and Milestones 8
3.3 Roles and Responsibilities 8
3.4 Risk Management Process 9

4 Deliverables **10**
4.1 Interim Deliverables 10
4.2 Final Deliverables 10

5 Communication and Coordination with Sponsor **11**
5.1 Communication Plan 11
5.2 Information Flow 11
5.3 Special Topics 11

6 Team Qualifications **12**
6.1 Team Overview 12
6.2 Individual Qualifications 12

7 Conclusions **14**

A Appendix A: Supporting Materials **15**

DRAFT

B Appendix B: Glossary of Terms and Acronyms 16

DRAFT

1 Objectives

1.1 Project Objectives

This project aims to deliver a lightweight, execution-agnostic regression testing platform for scientific and high-performance computing workloads. The system is designed to operate across diverse compute environments without relying on site-specific infrastructure, schedulers, or cluster configurations. The primary objectives are:

- **Detect performance drift early and reliably** by executing solver-defined workloads and capturing key performance metrics.
- **Reduce manual testing overhead** through automated, repeatable test execution and standardized logging.
- **Support heterogeneous HPC environments** by allowing solver scripts to define their own execution logic, independent of schedulers, resource managers, or hardware topology.
- **Provide clear, accessible performance insights** through a lightweight web interface that visualizes recent runs, historical trends, and detailed metrics.

1.2 Scope and Constraints

The project scope includes the design and implementation of:

- A modular framework for defining resources, systems, solvers, and tests.
- An execution-agnostic test runner that captures logs and extracts performance metrics.
- A lightweight results database and a web interface for scheduling and visualization.
- Example solvers and tests demonstrating end-to-end functionality.

The following items are explicitly out of scope:

- Production-grade test suites or integration with any specific organization's internal HPC environment.
- Full CI/CD[2] automation or cluster orchestration capabilities.

[2]Continuous Integration and Continuous Deployment; automated workflows that build, test, and deploy software to ensure rapid, reliable iteration.

DRAFT

- Hardware diagnostics beyond detecting solver-level performance drift.

1.3 Expected Outcomes and Deliverables

The project will produce a functional prototype capable of running end-to-end regression tests, storing results, and presenting performance trends. Expected outcomes include:

- A working regression-testing platform with modular configuration.
- A web-based dashboard for viewing recent and historical performance.
- Comprehensive user and developer documentation.
- Example solvers and tests demonstrating the system's capabilities.
- Final presentations for the sponsor and the MSSE[3] program.

1.4 Value Proposition of Deliverables to Sponsor

A general-purpose HPC regression testing framework provides significant value to the sponsor in several ways:

- **Substantial time savings** by automating one of the most labor-intensive aspects of maintaining scientific software.
- **Improved research and development velocity** by reducing blockers associated with running and validating solver software.
- **Lower operational costs** through early detection of performance regressions and the ability to direct resources toward resolving or optimizing underlying issues.
- **Enhanced community engagement** as an open-source tool that can attract contributors, foster collaboration, and support the development of future HPC talent.

2 Technical Approach

2.1 Overall Design Process

The project follows a structured, iterative design process that moves from requirements intake to architecture definition, prototype development, validation, and refinement. Early

[3] Master of Molecular Science and Software Engineering program at UC Berkeley.

DRAFT

stages focus on clarifying sponsor needs, defining system boundaries, and establishing a modular architecture for resources, systems, solvers, and tests. Development proceeds in a build–validate–iterate cycle, with weekly sponsor and faculty check-ins to confirm assumptions and ensure alignment. Validation criteria are defined alongside architectural decisions, enabling each component—framework, runner, database, and web interface—to be developed incrementally and evaluated through targeted tests before integration into an end-to-end MVP.

2.2 Concept Generation

Multiple architectural concepts were considered for the regression-testing platform. Alternatives included: (1) a scheduler-integrated system tightly coupled to SLURM[4], (2) a containerized execution environment using Docker or Singularity, and (3) an execution-agnostic framework where solver scripts define their own run logic. The team selected the execution-agnostic model because it supports diverse HPC environments, avoids assumptions about cluster configuration, and allows solvers to remain self-contained.

Additional concept exploration included database technologies (SQLite[5], Postgres[6], and analytical columnar engines such as DuckDB[7]), UI[8] frameworks, and approaches to defining solver metadata. Column-oriented databases are increasingly used in machine learning and scientific computing due to their high-performance column access patterns, which make them well suited for analytical queries over time-series performance metrics. The final concept emphasizes modularity, maintainability, and minimal operational overhead.

2.3 Analysis and Modeling

Analysis focuses on defining and extracting performance metrics that indicate solver stability and drift. Tools used include Python for log parsing and metric extraction, lightweight analytical databases for storing results, and web technologies for visualization. Performance metrics vary by solver but may include time-to-solution, MLUPS[9], accuracy, memory usage, and other domain-specific indicators.

[4] A widely used open-source workload manager for scheduling jobs on HPC clusters.
[5] A lightweight, file-based relational database engine.
[6] PostgreSQL, an advanced open-source relational database management system.
[7] An in-process analytical database optimized for columnar access patterns, widely used in ML and scientific workflows.
[8] User Interface; the visual and interactive components of a software system.
[9] Millions of Lattice Updates Per Second; a performance metric commonly used in computational physics and simulation codes.

DRAFT

Modeling efforts include defining expected performance ranges, establishing statistical baselines through repeated runs, and identifying thresholds for anomaly detection. Validation is integrated throughout this process: metrics are cross-checked against solver-defined expectations, anomaly thresholds are tested against known-good and known-bad cases, and repeated execution ensures that the platform can reliably detect deviations across heterogeneous environments.

2.4 Experimental Methods

Evaluation relies on running controlled test scenarios using open-source solvers and representative workloads. Each solver includes a definition file specifying allowed systems, success criteria, expected performance ranges, and log file formats. Tests may be scheduled or run on demand, with the platform passing environment variables and capturing all logs and outputs.

Experimental procedures include repeated runs to establish baseline performance, comparison of results across simulated system updates, and validation of anomaly detection logic. The web interface provides drill-down views for examining individual runs and historical trends, supporting both rapid debugging and long-term performance monitoring. Validation activities are documented through weekly QA[10] logs, ensuring traceability of assumptions, test coverage, and observed system behavior.

3 Project Management

3.1 Work Breakdown and Phases

The project follows a structured, phased workflow that moves from early planning through system design, implementation, testing, and final delivery. The major phases are:

- **Planning and Requirements (Weeks 1–3):** Sponsor kickoff, requirements intake, project charter, architecture sketch, and scope finalization.
- **Concept and System Design (Weeks 1–3):** Definition of resources, systems, solvers, and tests; validation strategy; documentation framework setup.
- **Detailed Design and MVP Build (Weeks 4–10):** Implementation of schemas, test runner prototype, log parsing, database setup, UI prototype, and sample solvers.

[10] Quality Assurance; systematic processes for verifying correctness, reliability, and performance.

DRAFT

- **Testing and Iteration (Weeks 11–13):** Internal test runs, sponsor validation cycle, UI/UX[11] refinement, metrics refinement, and system improvements.
- **Final Delivery (Weeks 14–16):** Documentation polish, final report, poster, repository cleanup, and final presentations.

This phased structure ensures predictable progress and allows for continuous sponsor feedback throughout the development cycle.

3.2 Schedule and Milestones

The project schedule is organized around a detailed Gantt[12] chart that spans January through May 2026. Key milestones include:

- **Course Kickoff** (Jan 21)
- **Flash demo[13] #1** (Feb 11)
- **Milestone Review I** (Feb 25)
- **Flash Demo #2** (Mar 11)
- **Milestone Review II** (Mar 31)
- **Flash Demo #3** (Apr 8)
- **Milestone Review III** (Apr 21)
- **Final Capstone Presentations** (May 6)

These milestones align with the build–validate–iterate cycle and ensure that the MVP is demonstrated, reviewed, and refined at regular intervals.

3.3 Roles and Responsibilities

The team is structured to balance leadership, technical development, validation, and documentation:

[11] User Experience; the overall usability, clarity, and interaction quality of a system from the user's perspective.
[12] See Appendix A for Gantt chart.
[13] Short academic demonstrations presented to the MSSE program.

DRAFT

- **Shree Patel — Project Lead:** Scheduling, timekeeping, communication, and coordination across phases.
- **Shawn Schulz — Tech Lead:** Architecture, schema design, sprint planning, and technical decision-making.
- **Kayleen Nordyke — QA/Validation Lead:** Testing strategy, validation notes, regression criteria, and metric refinement.
- **Brent Thorne — Docs & DevOps Lead:** Repository structure, documentation, formatting, database setup, and UI concept development.

This distribution ensures clear ownership while supporting collaborative development across all components of the platform.

3.4 Risk Management Process

Risk management is an ongoing process throughout the project lifecycle. The team identifies risks early, validates assumptions with the sponsor, and revisits risks at each milestone. Key elements of the process include:

- **Early Identification:** Risks related to solver definitions, access to HPC environments, and performance variability are documented during planning.
- **Assessment and Prioritization:** Risks are evaluated based on likelihood and impact, following PRAM[14]-aligned practices.
- **Mitigation Strategies:** Use of open-source solvers, clear scope-of-work, repeatable test procedures, and early integration of system components.
- **Ongoing Monitoring:** Weekly sponsor and faculty meetings provide continuous feedback loops to surface new risks and adjust plans accordingly.

This structured approach ensures that risks are proactively managed and that the project remains on track for successful delivery.

[14]Parallel Random Access Machine; a theoretical model for analyzing parallel algorithms.

DRAFT

4 Deliverables

4.1 Interim Deliverables

Throughout the semester, the team will produce a series of interim deliverables that demonstrate progress, validate assumptions, and provide visibility into system development. These include:

- **Project Scope and Charter** outlining requirements, constraints, and initial architecture.
- **Architecture Sketches and Schemas** for resources, systems, solvers, and tests.
- **Prototype Components** including the test runner, log parser, database schema, and early UI concepts.
- **Sample Solvers and Tests** used to validate the framework and establish baseline performance.
- **FLASH Demo Presentations** at Weeks 4, 8, and 12 demonstrating incremental functionality.
- **Milestone Review Materials** including videos, slides, and progress summaries.
- **Draft Documentation** covering user workflows, developer guidelines, and validation strategy.

4.2 Final Deliverables

At the conclusion of the project, the team will deliver a complete, end-to-end prototype of the regression-testing platform along with all required documentation and presentation materials. Final deliverables include:

- **Functional Prototype Platform** with resource/system/solver/test definitions, execution-agnostic runner, log capture, metric extraction, and results storage.
- **Web-Based Dashboard** for scheduling tests, viewing recent results, and exploring historical performance trends.
- **Comprehensive Documentation** including a user guide, developer guide, example solvers and tests, and contribution guidelines.

DRAFT

- **Final Report** summarizing design decisions, implementation details, validation results, and future recommendations.
- **Poster and Final Presentations** for the sponsor and the MSSE program.
- **Public GitHub Repository** containing the full codebase, documentation, and release materials.

5 Communication and Coordination with Sponsor

5.1 Communication Plan

The team maintains a consistent communication cadence to ensure alignment with Dow's expectations and to surface questions or risks early. Weekly sponsor meetings occur on Wednesdays at 9:00 AM PT and focus on requirement clarification, progress updates, and validation of design decisions. A weekly meeting with the faculty mentor is held on Thursdays at 9:00 AM PT to review technical direction and academic expectations. Internal team meetings take place on Mondays and Fridays to coordinate tasks, review progress, and prepare for sponsor and mentor discussions. Major milestones, FLASH demos, and review checkpoints provide structured opportunities for formal reporting and feedback.

5.2 Information Flow

Information is shared through clearly defined channels to ensure transparency and traceability. Sponsor-facing updates, questions, and design artifacts are delivered via email, Teams, and weekly meetings. Technical documentation, schemas, and implementation details are maintained in the project repository and wiki. Issue tracking, task assignments, and development coordination occur through GitHub Issues and internal communication platforms such as Discord. The faculty mentor receives weekly summaries of progress, risks, and upcoming work. All team members have access to the full set of project materials to support collaborative development.

5.3 Special Topics

No NDAs[15] or confidentiality agreements are required for this project, as all development uses open-source solvers and publicly shareable materials. The project does not involve

[15] Non-Disclosure Agreements; legal contracts restricting the sharing of confidential information.

DRAFT

handling proprietary Dow data or accessing internal HPC systems. No equipment loans are anticipated. All deliverables, including code, documentation, and examples, will be published in a public GitHub repository unless otherwise requested by the sponsor.

6 Team Qualifications

The project team brings a balanced combination of scientific domain knowledge, software engineering experience, and practical exposure to HPC workflows. Collectively, the team has experience in computational science, laboratory automation, bioinformatics, semiconductor research, and modern software development practices. This breadth enables the group to design and implement an execution-agnostic regression-testing platform that is technically sound, maintainable, and aligned with the needs of diverse HPC environments. The team's complementary skill sets position them to deliver a high-quality, on-time prototype that enhances testing capabilities for Dow and can be extended to broader scientific computing contexts.

6.1 Team Overview

As a group, Team DOW-1-26 combines:

- Strong foundations in chemistry, molecular biology, and scientific computing.
- Experience with HPC resource optimization, solver workflows, and performance analysis.
- Software engineering expertise spanning pipeline development, automation, and systems programming.
- Practical experience leading technical projects, coordinating cross-functional work, and producing clear technical documentation.

This combination of scientific insight and engineering capability supports the development of a modular, execution-agnostic testing framework that integrates smoothly into existing HPC workflows.

6.2 Individual Qualifications

Shree Patel — Project Lead. Shree holds a B.S. in Chemistry from Carnegie Mellon and has experience in HPC resource optimization for molecular dynamics, laboratory automation,

DRAFT

drug discovery, and AI product development for clinical trials. She is currently building Orbitl, a platform for modern scientific discovery, and brings strong leadership and systems-thinking skills to the project.

Kayleen Nordyke — QA/Validation Lead. Kayleen earned her B.S. in Chemistry from Purdue University and brings research experience in drug development and semiconductor materials. She contributes a structured approach to validation, testing, and quality assurance, supported by a strong interest in emerging AI technologies.

Shawn Schulz — Tech Lead. Shawn holds a B.A. in Molecular and Cell Biology from UC Berkeley and has worked in bioinformatics pipeline development at Stanford Medicine and software engineering at Ultima Genomics. His background in scientific software, hardware tinkering, and development of applications for HPC systems supports the technical architecture of the platform.

Brent Thorne — Docs & DevOps Lead. Brent has a background spanning electronics engineering, scientific computing, and high-performance systems, with experience in semiconductor devices, OS and driver development, and computational science. He brings deep expertise in HPC, mathematical modeling, and technical documentation, ensuring clarity, maintainability, and reproducibility across the project.

DRAFT

7 Conclusions

Dow, as the project sponsor, operates in a scientific computing environment where maintaining stable solver performance is essential for research productivity and operational reliability. Frequent software, security, and system updates introduce natural variability and, without an automated mechanism to detect regressions, both researchers and the HPC team face an increased risk of delays, inefficiencies, and reduced confidence in system behavior. This challenge motivates the development of a lightweight and execution-agnostic regression testing platform that supports Dow's broader goals of reliability, transparency, and operational efficiency.

The proposed solution—a modular framework for defining resources, systems, solvers, and tests—provides a practical and maintainable approach to identifying performance drift across diverse workloads. By standardizing the configuration and capturing logs and metrics through a unified runner, the platform enables early detection of anomalies and clear visualization of historical trends. The accompanying web interface further enhances accessibility for both developers and system operators.

For Dow, the benefits include reduced manual testing burden, improved reliability during system updates, and a long-term foundation for performance monitoring that can evolve with future needs. Because the system is execution-agnostic and is built on open-source components, it remains compatible with a wide range of HPC environments, including but not limited to Dow's internal systems.

The project team is well positioned to deliver this solution. With clearly defined roles across project leadership, technical architecture, validation strategy, and documentation, the team brings a balanced and coordinated approach to design, implementation, and communication. Through structured project management and regular engagement with Dow, the team will deliver a functional, maintainable prototype that addresses the sponsor's immediate needs while establishing a foundation for future expansion and integration.

DRAFT

A Appendix A: Supporting Materials

This appendix contains supplemental materials referenced in the proposal, including the project schedule and Gantt chart.

A.1 Project Schedule (Gantt Chart)

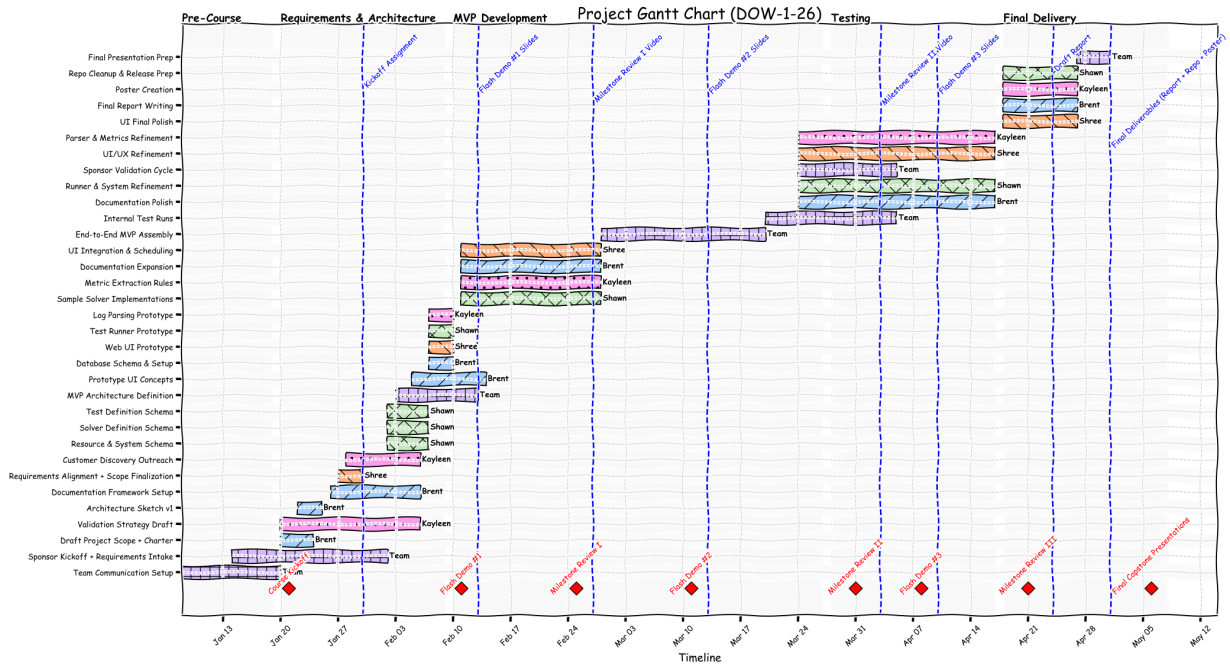

Figure 1: Project Schedule and Milestones

B Appendix B: Glossary of Terms and Acronyms

CI/CD Continuous Integration and Continuous Deployment; automated workflows that build, test, and deploy software to ensure rapid, reliable iteration.

DuckDB An in-process analytical database optimized for columnar access patterns, widely used in ML and scientific workflows.

Flash demos Short academic demonstrations presented to the MSSE program.

GitHub A platform for version control, collaboration, and software development using Git.

HPC High-Performance Computing; large-scale computational systems used for scientific and engineering workloads.

MLUPS Millions of Lattice Updates Per Second; a performance metric commonly used in computational physics and simulation codes.

MSSE Master of Molecular Science and Software Engineering program at UC Berkeley.

NDAs Non-Disclosure Agreement; a legal contract restricting the sharing of confidential information.

Postgres PostgreSQL, an advanced open-source relational database management system.

PRAM Parallel Random Access Machine; a theoretical model for analyzing parallel algorithms.

QA Quality Assurance; systematic processes for verifying correctness, reliability, and performance.

SQLite A lightweight, file-based relational database engine.

SLURM A widely used open-source workload manager for scheduling jobs on HPC clusters.

UI User Interface; the visual and interactive components of a software system.

UX User Experience; the overall usability, clarity, and interaction quality of a system from the user's perspective.

DRAFT